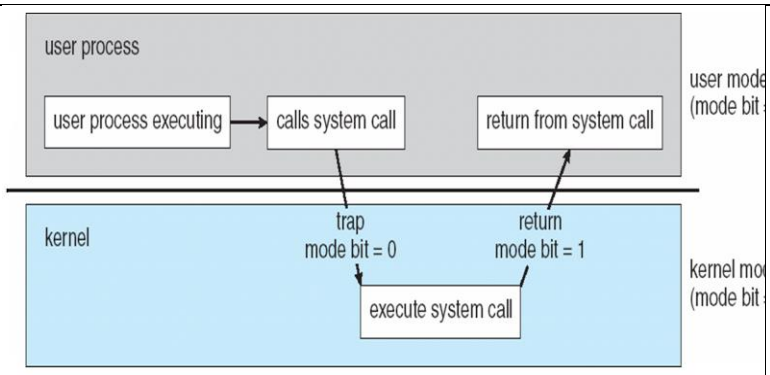
MANIPAL ACADEMY OF HIGHER EDUCATION

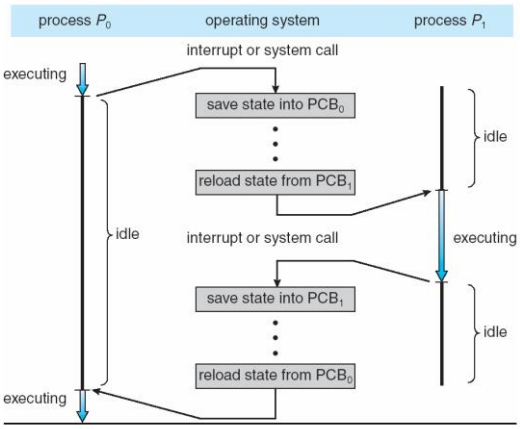
Mid-term Examination

Operating systems [CSE 3123] Scheme

Q. No		M	CLO	AHEP LO	BL
1	Identify the components of a program from the following list which are shared across threads in a multithreaded process. i) Register values ii) Heap memory iii) Global variable iv) Stack Memory a) i, ii b) i, ii ,iii **c) ii, iii** d) i, ii, iii, iv	0.5	1	1	2
2	Which multithreading model allows true concurrency on multiprocessor systems? a) Many-to-One b) **One-to-One** c) One-to-Many d) Cooperative model	0.5	1	1	2
3	Identify which scheduling algorithm suffers from the "convoy effect"? a) Round Robin **b) FCFS** c) Priority Scheduling d) Shortest Job First	0.5	3	2	2
4	There are 10 different processes running on a workstation. Idle processes are waiting for an input event in the input queue. Busy processes are scheduled with the Round-Robin time sharing method. Which out of the following quantum times is the best value for small response times, if the processes have a short runtime, e.g. less than 10ms? a) **tQ = 15ms** b) tQ = 40ms c) tQ = 45ms d) tQ = 50ms	0.5	3	2	3
5	Select a solution to the problem of indefinite blocking of low-priority processes. a) Starvation **b) Aging** c) Convoy effect d) Spinlock	0.5	3	2	2
6	Deadlock can be modeled using a) State transition diagram b**) Resource Allocation Graph** c) Resource Request Graph d) Wait-for-graph	0.5	3	2	2
7	In which binding scheme can a process be moved during execution?	0.5	CO 4	2	L2

	a) Compile-time binding b) Load-time binding c**) Execution-time binding** d) Static binding				
8	**Swapping is primarily used to** a) Increase CPU speed **b) Increase multiprogramming degree** c) Reduce external fragmentation d) Handle deadlocks	0.5	CO 4	2	L2
9	**Which of the following allocation strategies can cause external fragmentation?** a) Paging b) Segmentation **c) Contiguous allocation** d) None of these	0.5	CO 4	2	L2
10	Which of the following is NOT typically supported in mobile systems? a) Paging **b) Swapping** c) Dynamic loading d) Contiguous allocation	0.5	CO 4	2	L2
11	With a neat diagram, elaborate on how the operating system distinguishes between the execution of operating system code and user-defined code **Diagram -1M, Explanation -2M** By using atleast two separate modes of operation: user mode and kernel mode (also called supervisor mode, system mode, or privileged mode). A bit, called the mode bit, is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1). With the mode bit, we can distinguish between a task that is executed on behalf of the operating system and one that is executed on behalf of the user. When the computer system is executing on behalf of a user application, the system is in user mode. However, when a user application requests a service from the operating system (via a system call), the system must transition from user to kernel mode to fulfill the request. At system boot time, the hardware starts in kernel mode. The operating system is then loaded and starts user applications in user mode. Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode (that is, changes the state of the mode bit to 0). Thus, whenever the operating system gains control of the computer, it is in kernel mode. The system always switches to user mode (by setting the mode bit to 1) before passing control to a user program.	3M	1	1	3

					
12	Differentiate between direct communication and indirect communication. Mention all the properties of communication link in both type of communication. Ans: Definition 1M, Properties 1M each **Direct Communication** • Definition: Processes must explicitly name each other to communicate. Example: send(P, message) → send to process P; receive(Q, message) → receive from process Q. **Properties of communication link (Direct):** 1. Link is established automatically between processes that want to communicate. 2. Exactly one link exists between every pair of communicating processes. 3. Link is bi-directional (can be used by both processes for communication). 4. Processes must know each other's identity explicitly (tight coupling). --- **2. Indirect Communication** • Definition: Processes communicate via a shared mailbox (or port). The mailbox is an intermediate communication object. Example: send(A, message) → send message to mailbox A; receive(A, message) → receive message from mailbox A. **Properties of communication link (Indirect):** 1. Link is established between processes only if they share a common mailbox. 2. Multiple processes can share the same mailbox (many-to-many communication). 3. Link can be unidirectional or bi-directional.	3M	1	1	4

	4. Processes need not know each other's identity explicitly (loose coupling; they just refer to the mailbox).				
13	Illustrate and explain with a diagram how the CPU switches from a process to a process. Compare and contrast monolithic and microkernel architectures. **Ans: Diagram -0.5M,Expalantion 1M Any three Differences-1.5M**  MICROKERNEL KERNEL / MONOLITHIC KERNEL A kernel type that provides mechanisms such as low-level address space management, thread management and interprocess communication to implement an operating system / A type of kernel in operating systems where the entire operating system works in the kernel space OS services and kernel are separated / Kernel contains the OS services Slow / Fast Failure in one component will not affect the other components / Failure in one component will affect the entire system Easier to add new functionalities / Difficult to add new functionalities Smaller in size / Larger in size	3M	1	1	3
14	Apply semaphore operations (wait and signal) to show how they can be used to control access to a printer shared by multiple processes Ans: A **semaphore** is used to control access to a shared resource like a printer.	2	2	2	3

	• We initialize the **semaphore value = 1**, since only **one process** can use the printer at a time.
	• Each process must perform a **wait()** before using the printer and a **signal()** after finishing.
	Pseudo-code:
	Semaphore S = 1; // binary semaphore for printer
	
	Process:
	wait(S); // request access
	// critical section: use printer
	signal(S); // release access
	• **wait(S):** If S > 0, decrements S and allows the process to enter. If S = 0, process waits (blocked).
	• **signal(S):** Increments S, allowing another waiting process to access the printer.
	Thus, semaphore ensures **mutual exclusion**, preventing two processes from using the printer simultaneously.
	
	definition of semaphore and its use for mutual exclusio 1/2
	
	Explanation of wait() and signal() 1
	Application to printer scenario 1/2				

15.	Assume that the following processes are scheduled using a preemptive priority scheduling algorithm. Each process is assigned a numerical priority, with a smaller number indicating a higher relative priority. **Process** | **Arrival Time** | **Burst Time** | **Priority** P1 | 0 | 8 | 2 P2 | 1 | 4 | 1 P3 | 2 | 9 | 3 P4 | 3 | 5 | 2 1. Construct the **Gantt chart** showing how the CPU is allocated over time. 2. Calculate the **Turnaround Time** , and **Waiting Times** for each process. 3. Find the **Average Waiting Time (AWT)** and **Average Turnaround Time (ATAT)**. Ans: **Gantt Chart= 1M TAT= 1M WT =1M Avg WT and TAT 0.5M** 0 —1— 1——5—— 5————12———— 12——17—— 17————26 |P1| | P2 | P1 | P4 | P3 **Step-by-step scheduling (timeline reasoning)**	4	3	2	4

<table>
<tr>
<td></td>
<td>

t = 0 → 1: Only P1 has arrived, so P1 runs for 1 unit.

P1 remaining = 8 − 1 = 7.

t = 1: P2 arrives (priority 1), which is higher than P1 (priority 2). P2 preempts P1.
t = 1 → 5: P2 runs (burst 4) with no higher-priority arrivals in between.

P2 finishes at t = 5.

At t = 5 the ready set is: P1 (rem 7, pr 2, arrived 0), P3 (9, pr 3, arrived 2), P4 (5, pr 2, arrived 3).
Highest priority among ready is priority 2 (P1 and P4).
Tie → FCFS: P1 (arrived earlier) goes next.
t = 5 → 12: P1 runs its remaining 7 units and finishes at t = 12.
t = 12 → 17: Next highest ready is P4 (priority 2). P4 runs full 5 units and finishes at t = 17.
t = 17 → 26: Finally P3 (priority 3) runs for its 9 units and finishes at t = 26.

</td>
<td></td>
<td></td>
<td></td>
<td></td>
</tr>
<tr>
<td>16</td>
<td>
A computer system has 5 processes (P1–P5) competing for 3 resource types (A, B, C).
Apply Banker’s algorithm to do compute the following tasks.

<table>
<tr><th>Processes</th><th>Allocation</th><th>Max</th><th>Available</th></tr>
<tr><td></td><td>A B C</td><td>A B C</td><td>A B C</td></tr>
<tr><td>P1</td><td>1 0 2</td><td>3 2 2</td><td>2 1 2</td></tr>
<tr><td>P2</td><td>2 1 0</td><td>4 2 2</td><td></td></tr>
<tr><td>P3</td><td>3 1 2</td><td>6 3 4</td><td></td></tr>
<tr><td>P4</td><td>0 2 1</td><td>2 3 3</td><td></td></tr>
<tr><td>P5</td><td>1 1 1</td><td>4 2 3</td><td></td></tr>
</table>
(a) Construct the Need Matrix for all processes.
(b) Apply the Safety Algorithm to check if the system is currently in a safe state. Find the safe sequence.
(c) If Process P1 makes a request (1,0,1), check whether the request can be granted immediately.

Ans: (a) Need matrix = Max – Allocation 1M
Process Need (A,B,C)

P1 (3−1, 2−0, 2−2) = (2,2,0)

P2 (4−2, 2−1, 2−0) = (2,1,2)

P3 (6−3, 3−1, 4−2) = (3,2,2)

P4 (2−0, 3−2, 3−1) = (2,1,2)
</td>
<td>4</td>
<td>3</td>
<td>2</td>
<td>4</td>
</tr>
</table>

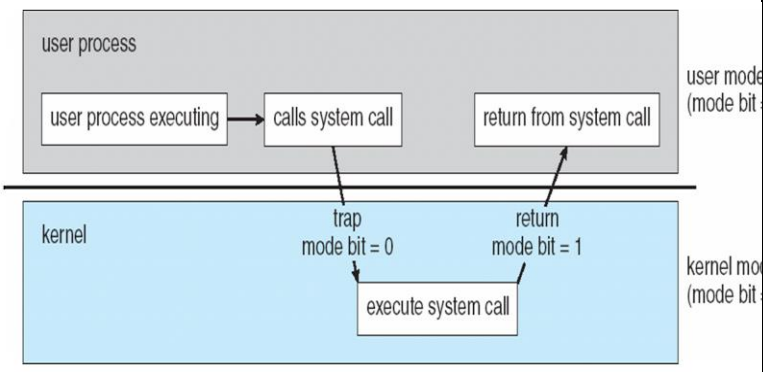
	P5 (4−1, 2−1, 3−1) = **(3,1,2)** **(b) Safety algorithm (is the state safe?) 2M** Start Work = Available = (2,1,2), Finish = {}. Find a process whose **Need ≤ Work**: • P1 need (2,2,0): B need 2 > Work.B 1 → **cannot** run now. • P2 need (2,1,2): 2≤2, 1≤1, 2≤2 → **can run**. Run **P2**, add Allocation(P2)=(2,1,0) to Work → Work = (4,2,2); mark P2 finished. With Work = (4,2,2): • P1 need (2,2,0) fits → run **P1**, add Allocation(P1)=(1,0,2) → Work = (5,2,4); mark P1 finished. • P3 need (3,2,2) fits → run **P3**, add Allocation(P3)=(3,1,2) → Work = (8,3,6); mark P3 finished. • P4 need (2,1,2) fits → run **P4**, add Allocation(P4)=(0,2,1) → Work = (8,5,7); mark P4 finished. • P5 need (3,1,2) fits → run **P5**, add Allocation(P5)=(1,1,1) → Work = (9,6,8); mark P5 finished. All processes can finish. **Therefore the system is SAFE.** One safe sequence is: **P2 → P1 → P3 → P4 → P5**. (Any order that respects the Need≤Work checks is acceptable; the sequence above is valid.) **(c) Request by P1 = (1,0,1). Can it be granted immediately? 1M** First check against P1's Need: • Request (1,0,1) ≤ Need(P1) (2,2,0)? Compare component-wise: A:1≤2 ✓, B:0≤2 ✓, C:1≤0 ✗ — **NO**, request for C (1) **exceeds** P1's declared need for C (0). Because the request exceeds the process's remaining declared need, **the request must be denied** (it is invalid under the Banker's model). We do **not** proceed to the safety test.				
17.	Apply the different dynamic storage allocation strategies (first-fit, best-fit, worst-fit) to allocate memory holes of sizes 200 KB, 300 KB, and 100 KB to four processes arriving with memory requirements of P1 = 180 KB, P2 = 260 KB, P3 = 100 KB, P4 = 210 KB. Show which strategy is most efficient Ans: 1M each First-Fit (scan left → right)	3M	4	3	3

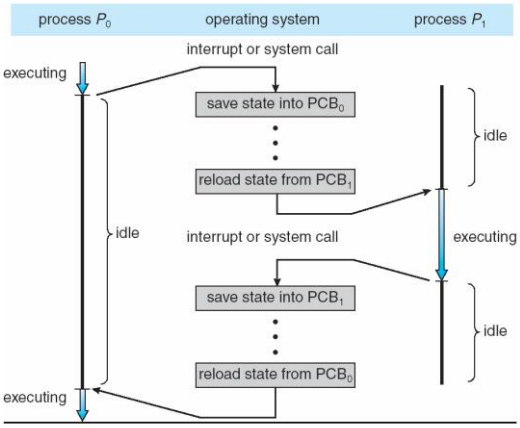
	1. **P1 (180)** → H1 = 200 fits → allocate 180, leftover H1 = **20**. Holes → [20, 300, 100] 2. **P2 (260)** → 20 no, 300 fits → allocate 260, leftover H2 = **40**. Holes → [20, 40, 100] 3. **P3 (100)** → 20 no, 40 no, 100 fits exactly → allocate 100, leftover H3 = **0** (hole used up). Holes → [20, 40] 4. **P4 (210)** → 20 no, 40 no → **cannot allocate**. **Result (First-Fit):** P1, P2, P3 allocated; P4 fails. Remaining holes: 20 KB, 40 KB. Best-Fit (choose smallest hole ≥ request) 1. **P1 (180)** → candidates: 200,300 → best = 200 → allocate 180, leftover **20**. Holes → [20, 300, 100] 2. **P2 (260)** → candidates: 300 → allocate 260, leftover **40**. Holes → [20, 40, 100] 3. **P3 (100)** → candidate: 100 → allocate 100, leftover **0**. Holes → [20, 40] 4. **P4 (210)** → 20 no, 40 no → **cannot allocate**. **Result (Best-Fit):** P1, P2, P3 allocated; P4 fails. Remaining holes: 20 KB, 40 KB. *(For this instance Best-Fit behaves the same as First-Fit.)* Worst-Fit (choose largest hole that fits) 1. **P1 (180)** → largest hole = 300 → allocate 180, leftover H2 = **120**. Holes → [200, 120, 100] 2. **P2 (260)** → check largest hole(s): 200,120,100 — **none ≥ 260 → cannot allocate P2**. **Result (Worst-Fit):** Allocation fails at P2. Only P1 allocated; P2, P3, P4 not allocated. **First fit and best fit are most efficient**				
18.	Apply the idea of fragmentation to explain how compaction can reduce external fragmentation, and why it cannot eliminate internal fragmentation. Ans: **Sample Answer**	3M	4	3	3

	• **External fragmentation** occurs when free memory is available but scattered in small non-contiguous blocks. • **Compaction** is a technique where the OS shuffles allocated processes in memory so that all free memory is collected together into a single large block. ○ This reduces **external fragmentation**, since large processes can now be allocated in the consolidated free space. • **Internal fragmentation** occurs when allocated memory is slightly larger than requested, leaving unused space inside the allocated block. • **Compaction cannot eliminate internal fragmentation** because this wasted space lies **within the allocated partitions themselves**, not in scattered free blocks. **Mark Split (3 Marks)** **Component** — **Marks** External fragmentation concept + compaction definition — 1 Internal fragmentation concept — 1 **Compaction cannot eliminate internal** — 1				

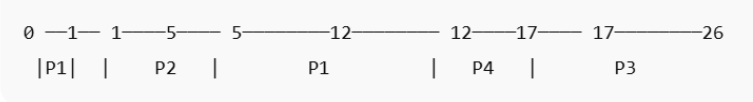
Q. No	MANIPAL ACADEMY OF HIGHER EDUCATION OPERATING SYSTEMS [CSE 3123] Marks: 30 Duration: 90 mins	M	CLO	AHEP LO	BL
1	Identify the components of a program from the following list which are shared across threads in a multithreaded process. i) Register values ii) Heap memory iii) Global variable iv) Stack Memory e) i, ii f) i, ii ,iii **g) ii, iii** h) i, ii, iii, iv	0.5	1	1	2
2	Which multithreading model allows true concurrency on multiprocessor systems? a) Many-to-One b) **One-to-One** c) One-to-Many d) Cooperative model	0.5	1	1	2
3	Identify which scheduling algorithm suffers from the "convoy effect"? a) Round Robin **b) FCFS** c) Priority Scheduling d) Shortest Job First	0.5	3	2	2
4	There are 10 different processes running on a workstation. Idle processes are waiting for an input event in the input queue. Busy processes are scheduled with the Round-Robin time sharing method. Which out of the following quantum times is the best value for small response times, if the processes have a short runtime, e.g. less than 10ms? a) **tQ = 15ms** b) tQ = 40ms c) tQ = 45ms d) tQ = 50ms	0.5	3	2	3
5	Select a solution to the problem of indefinite blocking of low-priority processes. e) Starvation **f) Aging** g) Convoy effect h) Spinlock	0.5	3	2	2
6	Deadlock can be modeled using a) State transition diagram b**) Resource Allocation Graph** c) Resource Request Graph d) Wait-for-graph	0.5	3	2	2

7	In which binding scheme can a process be moved during execution? a) Compile-time binding b) Load-time binding c**) Execution-time binding** d) Static binding	0.5	CO4	2	L2
8	**Swapping is primarily used to** a) Increase CPU speed **b) Increase multiprogramming degree** c) Reduce external fragmentation d) Handle deadlocks	0.5	CO4	2	L2
9	**Which of the following allocation strategies can cause external fragmentation?** a) Paging b) Segmentation **c) Contiguous allocation** d) None of these	0.5	CO4	2	L2
10	Which of the following is NOT typically supported in mobile systems? a) Paging **b) Swapping** c) Dynamic loading d) Contiguous allocation	0.5	CO4	2	L2
11	With a neat diagram, elaborate on how the operating system distinguishes between the execution of operating system code and user-defined code **Diagram -1M, Explanation -2M** By using atleast two separate modes of operation: user mode and kernel mode (also called supervisor mode, system mode, or privileged mode). A bit, called the mode bit, is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1). With the mode bit, we can distinguish between a task that is executed on behalf of the operating system and one that is executed on behalf of the user. When the computer system is executing on behalf of a user application, the system is in user mode. However, when a user application requests a service from the operating system (via a system call), the system must transition from user to kernel mode to fulfill the request. At system boot time, the hardware starts in kernel mode. The operating system is then loaded and starts user applications in user mode. Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode (that is, changes the state of the mode bit to 0). Thus, whenever the operating system gains control of the computer, it is in kernel mode. The system always switches to user mode (by setting the mode bit to 1) before passing control to a user program.	3M	1	1	3

					
12	Differentiate between direct communication and indirect communication. Mention all the properties of communication link in both type of communication. Ans: Definition 1M, Properties 1M each **Direct Communication** • Definition: Processes must explicitly name each other to communicate. Example: send(P, message) → send to process P; receive(Q, message) → receive from process Q. **Properties of communication link (Direct):** 5. Link is established automatically between processes that want to communicate. 6. Exactly one link exists between every pair of communicating processes. 7. Link is bi-directional (can be used by both processes for communication). 8. Processes must know each other's identity explicitly (tight coupling). --- **2. Indirect Communication** • Definition: Processes communicate via a shared mailbox (or port). The mailbox is an intermediate communication object. Example: send(A, message) → send message to mailbox A; receive(A, message) → receive message from mailbox A. **Properties of communication link (Indirect):** 5. Link is established between processes only if they share a common mailbox. 6. Multiple processes can share the same mailbox (many-to-many communication).	3M	1	1	4

	7. Link can be unidirectional or bi-directional. 8. Processes need not know each other's identity explicitly (loose coupling; they just refer to the mailbox).				
13	Illustrate and explain with a diagram how the CPU switches from a process to a process. Compare and contrast monolithic and microkernel architectures. **Ans: Diagram -0.5M,Expalantion 1M Any three Differences-1.5M**  MICROKERNEL KERNEL / MONOLITHIC KERNEL A kernel type that provides mechanisms such as low-level address space management, thread management and interprocess communication to implement an operating system / A type of kernel in operating systems where the entire operating system works in the kernel space OS services and kernel are separated / Kernel contains the OS services Slow / Fast Failure in one component will not affect the other components / Failure in one component will affect the entire system Easier to add new functionalities / Difficult to add new functionalities Smaller in size / Larger in size	3M	1	1	3
14	Apply semaphore operations (wait and signal) to show how they can be used to control access to a printer shared by multiple processes Ans: A **semaphore** is used to control access to a shared resource like a printer.	2	2	2	3

	• We initialize the **semaphore value = 1**, since only **one process** can use the printer at a time.
	• Each process must perform a **wait()** before using the printer and a **signal()** after finishing.
	Pseudo-code:
	Semaphore S = 1; // binary semaphore for printer
	
	Process:
	wait(S); // request access
	// critical section: use printer
	signal(S); // release access
	• **wait(S):** If S > 0, decrements S and allows the process to enter. If S = 0, process waits (blocked).
	• **signal(S):** Increments S, allowing another waiting process to access the printer.
	Thus, semaphore ensures **mutual exclusion**, preventing two processes from using the printer simultaneously.
	
	definition of semaphore and its use for mutual exclusio 1/2
	
	Explanation of wait() and signal() 1
	Application to printer scenario 1/2				

15.	Assume that the following processes are scheduled using a preemptive priority scheduling algorithm. Each process is assigned a numerical priority, with a smaller number indicating a higher relative priority. **Process** | **Arrival Time** | **Burst Time** | **Priority** P1 | 0 | 8 | 2 P2 | 1 | 4 | 1 P3 | 2 | 9 | 3 P4 | 3 | 5 | 2 4. Construct the **Gantt chart** showing how the CPU is allocated over time. 5. Calculate the **Turnaround Time** , and **Waiting Times** for each process. 6. Find the **Average Waiting Time (AWT)** and **Average Turnaround Time (ATAT)**. Ans: **Gantt Chart= 1M TAT= 1M WT =1M Avg WT and TAT 0.5M**  **Step-by-step scheduling (timeline reasoning)**	4	3	2	4

<table>
<tr>
<td></td>
<td>

t = 0 → 1: Only P1 has arrived, so P1 runs for 1 unit.

P1 remaining = 8 − 1 = 7.

t = 1: P2 arrives (priority 1), which is higher than P1 (priority 2). P2 preempts P1.
t = 1 → 5: P2 runs (burst 4) with no higher-priority arrivals in between.

P2 finishes at t = 5.

At t = 5 the ready set is: P1 (rem 7, pr 2, arrived 0), P3 (9, pr 3, arrived 2), P4 (5, pr 2, arrived 3).
Highest priority among ready is priority 2 (P1 and P4).
Tie → FCFS: P1 (arrived earlier) goes next.
t = 5 → 12: P1 runs its remaining 7 units and finishes at t = 12.
t = 12 → 17: Next highest ready is P4 (priority 2). P4 runs full 5 units and finishes at t = 17.
t = 17 → 26: Finally P3 (priority 3) runs for its 9 units and finishes at t = 26.

</td>
<td></td>
<td></td>
<td></td>
<td></td>
</tr>
<tr>
<td>16</td>
<td>
A computer system has 5 processes (P1–P5) competing for 3 resource types (A, B, C).
Apply Banker’s algorithm to do compute the following tasks.

<table>
<tr><th>Processes</th><th>Allocation</th><th>Max</th><th>Available</th></tr>
<tr><td></td><td>A B C</td><td>A B C</td><td>A B C</td></tr>
<tr><td>P1</td><td>1 0 2</td><td>3 2 2</td><td>2 1 2</td></tr>
<tr><td>P2</td><td>2 1 0</td><td>4 2 2</td><td></td></tr>
<tr><td>P3</td><td>3 1 2</td><td>6 3 4</td><td></td></tr>
<tr><td>P4</td><td>0 2 1</td><td>2 3 3</td><td></td></tr>
<tr><td>P5</td><td>1 1 1</td><td>4 2 3</td><td></td></tr>
</table>
(a) Construct the Need Matrix for all processes.
(b) Apply the Safety Algorithm to check if the system is currently in a safe state. Find the safe sequence.
(c) If Process P1 makes a request (1,0,1), check whether the request can be granted immediately.

Ans: (a) Need matrix = Max – Allocation 1M
Process Need (A,B,C)

P1 (3−1, 2−0, 2−2) = (2,2,0)

P2 (4−2, 2−1, 2−0) = (2,1,2)

P3 (6−3, 3−1, 4−2) = (3,2,2)

P4 (2−0, 3−2, 3−1) = (2,1,2)
</td>
<td>4</td>
<td>3</td>
<td>2</td>
<td>4</td>
</tr>
</table>

	P5 (4−1, 2−1, 3−1) = **(3,1,2)** **(b) Safety algorithm (is the state safe?) 2M** Start Work = Available = (2,1,2), Finish = {}. Find a process whose **Need ≤ Work**: • P1 need (2,2,0): B need 2 > Work.B 1 → **cannot** run now. • P2 need (2,1,2): 2≤2, 1≤1, 2≤2 → **can run**. Run **P2**, add Allocation(P2)=(2,1,0) to Work → Work = (4,2,2); mark P2 finished. With Work = (4,2,2): • P1 need (2,2,0) fits → run **P1**, add Allocation(P1)=(1,0,2) → Work = (5,2,4); mark P1 finished. • P3 need (3,2,2) fits → run **P3**, add Allocation(P3)=(3,1,2) → Work = (8,3,6); mark P3 finished. • P4 need (2,1,2) fits → run **P4**, add Allocation(P4)=(0,2,1) → Work = (8,5,7); mark P4 finished. • P5 need (3,1,2) fits → run **P5**, add Allocation(P5)=(1,1,1) → Work = (9,6,8); mark P5 finished. All processes can finish. **Therefore the system is SAFE.** One safe sequence is: **P2 → P1 → P3 → P4 → P5**. (Any order that respects the Need≤Work checks is acceptable; the sequence above is valid.) **(c) Request by P1 = (1,0,1). Can it be granted immediately? 1M** First check against P1's Need: • Request (1,0,1) ≤ Need(P1) (2,2,0)? Compare component-wise: A:1≤2 ✓, B:0≤2 ✓, C:1≤0 ✗ — **NO**, request for C (1) **exceeds** P1's declared need for C (0). Because the request exceeds the process's remaining declared need, **the request must be denied** (it is invalid under the Banker's model). We do **not** proceed to the safety test.				
17.	Apply the different dynamic storage allocation strategies (first-fit, best-fit, worst-fit) to allocate memory holes of sizes 200 KB, 300 KB, and 100 KB to four processes arriving with memory requirements of P1 = 180 KB, P2 = 260 KB, P3 = 100 KB, P4 = 210 KB. Show which strategy is most efficient Ans: 1M each First-Fit (scan left → right)	3M	4	3	3

	5. **P1 (180)** → H1 = 200 fits → allocate 180, leftover H1 = **20**. Holes → [20, 300, 100] 6. **P2 (260)** → 20 no, 300 fits → allocate 260, leftover H2 = **40**. Holes → [20, 40, 100] 7. **P3 (100)** → 20 no, 40 no, 100 fits exactly → allocate 100, leftover H3 = **0** (hole used up). Holes → [20, 40] 8. **P4 (210)** → 20 no, 40 no → **cannot allocate**. **Result (First-Fit):** P1, P2, P3 allocated; P4 fails. Remaining holes: 20 KB, 40 KB. Best-Fit (choose smallest hole ≥ request) 5. **P1 (180)** → candidates: 200,300 → best = 200 → allocate 180, leftover **20**. Holes → [20, 300, 100] 6. **P2 (260)** → candidates: 300 → allocate 260, leftover **40**. Holes → [20, 40, 100] 7. **P3 (100)** → candidate: 100 → allocate 100, leftover **0**. Holes → [20, 40] 8. **P4 (210)** → 20 no, 40 no → **cannot allocate**. **Result (Best-Fit):** P1, P2, P3 allocated; P4 fails. Remaining holes: 20 KB, 40 KB. *(For this instance Best-Fit behaves the same as First-Fit.)* Worst-Fit (choose largest hole that fits) 3. **P1 (180)** → largest hole = 300 → allocate 180, leftover H2 = **120**. Holes → [200, 120, 100] 4. **P2 (260)** → check largest hole(s): 200,120,100 — **none ≥ 260 → cannot allocate P2**. **Result (Worst-Fit):** Allocation fails at P2. Only P1 allocated; P2, P3, P4 not allocated. **First fit and best fit are most efficient**				
18.	Apply the idea of fragmentation to explain how compaction can reduce external fragmentation, and why it cannot eliminate internal fragmentation. Ans: **Sample Answer**	3M	4	3	3

	• **External fragmentation** occurs when free memory is available but scattered in small non-contiguous blocks. • **Compaction** is a technique where the OS shuffles allocated processes in memory so that all free memory is collected together into a single large block. ○ This reduces **external fragmentation**, since large processes can now be allocated in the consolidated free space. • **Internal fragmentation** occurs when allocated memory is slightly larger than requested, leaving unused space inside the allocated block. • **Compaction cannot eliminate internal fragmentation** because this wasted space lies **within the allocated partitions themselves**, not in scattered free blocks. **Mark Split (3 Marks)** **Component** — **Marks** External fragmentation concept + compaction definition — 1 Internal fragmentation concept — 1 **Compaction cannot eliminate internal** — 1				